

Scan Surprise

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

I've gotten bored of handing out flags as text. Wouldn't it be cool if they were an image instead?

Additional details will be available after launching your challenge instance.

Hints

- QR codes are a way of encoding data. While they're most known for storing URLs, they can store other things too.
 - Mobile phones have included native QR code scanners in their cameras since version 8 (Oreo) and iOS 11
 - If you don't have access to a phone, you can also use `zbar-tools` to convert an image to text
-

Tools Used

- **unzip:** Extract files from ZIP archives
 - **file:** Identify file types and basic information
 - **strings:** Extract printable strings from binary files
 - **grep:** Search for patterns in text output
 - **exiftool:** Examine and extract metadata from files
 - **zbarimg:** Decode QR codes and barcodes from images
-

Complete Solution

Step 1: Download and Prepare

Download the zip file named `challenge.zip` from the challenge page.

Once downloaded, extract the contents:

```
unzip challenge.zip
```

You'll find an image file in the extracted folder.



Step 2: Basic File Inspection

First, verify the file type and general information:

```
file home/ctf-player/drop-in/flag.png
```

Expected output:

```
home/ctf-player/drop-in/flag.png: PNG image data, 99 x 99, 1-bit colormap, non-interlaced
```

Step 3: Search for Visible Text

The flag may be present as plain text. Try extracting all character strings:

```
strings home/ctf-player/drop-in/flag.png | grep -i "picoCTF"
```

⚠ *If no results appear, proceed to the next step (decoding).*

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata:

```
exiftool home/ctf-player/drop-in/flag.png
```

Complete output:

```
ExifTool Version Number      : 12.76
File Name                    : flag.png
Directory                   : home/ctf-player/drop-in
```

```
File Size : 346 bytes
File Modification Date/Time : 2024:03:12 00:50:25+01:00
File Access Date/Time : 2026:05:08 13:41:33+01:00
File Inode Change Date/Time : 2026:05:08 13:41:33+01:00
File Permissions : -rwxrwxrwx
File Type : PNG
File Type Extension : png
MIME Type : image/png
Image Width : 99
Image Height : 99
Bit Depth : 1
Color Type : Palette
Compression : Deflate/Inflate
Filter : Adaptive
Interlace : Noninterlaced
Palette : (Binary data 6 bytes, use -b option to extract)
Transparency : 255 255
Pixels Per Unit X : 2834
Pixels Per Unit Y : 2834
Pixel Units : meters
Image Size : 99x99
Megapixels : 0.010
```

The metadata appears normal, but upon closer inspection of the image, you'll notice it's actually a QR code.

You have two options: scan it with your phone's camera or use the `zbarimg` command.

```
zbarimg home/ctf-player/drop-in/flag.png
```

Expected output:

```
QR-Code:picoCTF{<REDACTED>}
scanned 1 barcode symbols from 1 images in 0.02 seconds
```

 **Note:** The actual flag is redacted here. In the real challenge, you will see the complete flag.

Final Result

```
picoCTF{<REDACTED>}
```

Key Concepts

- **QR Code Decoding:** QR codes can encode various types of data beyond URLs, including text and flags.
- **Image Analysis:** Visual inspection of images can reveal hidden information like QR codes.
- **Metadata Examination:** Tools like exiftool help analyze file properties and embedded data.
- **Command-line Tools:** Utilities like zbarimg provide programmatic ways to decode image-based data.
- **File Type Identification:** Understanding file formats helps determine appropriate analysis methods.
- **Text Extraction:** Searching for strings within files can uncover embedded or hidden text content.